

Table of Contents

Unit 2: Programming and Problem Solving with Python.....	3
An 11th Class Computer Science Study Guide	3
Introduction.....	3
2.1 Introduction to Python Programming.....	3
2.1.1 Understanding Basic Programming Concepts.....	3
2.1.1.1 Programming Basics	3
2.1.1.2 Setting Up Python Development Environment	5
2.2 Basic Python Syntax and Structure	7
2.2.1 Variables, Data Types and Input/Output.....	8
2.2.1.1 Variable.....	8
2.2.1.2 Variable Naming Rules in Python	9
2.2.1.3 Creating Different Types of Variables	10
2.2.1.4 Input and Output Operations	11
2.2.1.5 Handling Integer and Float Inputs	12
2.3 Operators and Expressions.....	13
2.3.1 Arithmetic Operators.....	14
2.3.2 Comparison Operators	14
2.3.3 Assignment Operators	14
2.3.4 Logical Operators	15
2.3.5 Expressions	15
2.3.6 Operator Precedence in Python	15
2.4 Control Structures.....	16
2.4.1 Decision Making	17
2.4.1.1 If Statement	17
2.4.1.2 If-else Statement	17
2.4.1.3 Short Hand if-else Statement	18
2.4.1.4 if-elif-else Statement.....	18
2.4.2 Looping Constructs.....	19
2.4.2.1 while Loop	19
2.4.2.2 for Loop	20
2.5 Python Modules and Built-in Data Structures.....	22
2.5.1 Functions and Modules	22
2.5.1.1 Defining and Invoking Functions	22
2.5.1.2 Function Parameters and Return Values.....	23
2.5.1.3 Default Parameters.....	23

2.5.2 Using Libraries and Modules	24
2.5.3 Importing and Using Libraries.....	24
2.5.3.1 Package Structure	24
2.6 Built-in Data Structures.....	25
2.6.1 Lists	25
2.6.1.1 Creating, Accessing, and Modifying Lists	25
2.6.1.2 Accessing List Items	26
2.6.1.3 Modifying a List.....	26
2.6.1.4 Methods and Operations on Lists	26
2.6.1.5 List Operations	26
2.6.2 Tuples	28
2.6.3 Indexing and Slicing	28
2.6.3.1 Indexing	28
2.6.3.2 Slicing	28
2.6.3.3 Indexing and Slicing with Negative Indices	29
2.7 Modular Programming in Python	30
2.8 Object-Oriented Programming in Python	32
2.8.1 Class and Objects	32
2.8.1.1 Defining Classes and Creating Objects.....	33
2.9 Advanced Python Concepts	34
2.9.1 Exception Handling.....	34
2.9.1.1 Try-Except Blocks.....	35
2.9.2 File Handling.....	36
2.9.2.1 Opening, Reading, and Closing Files	36
2.9.2.2 Writing to Files	36
2.10 Testing and Debugging in Python.....	37
2.10.1 Testing	37
2.10.1.1 Types of Testing	38
2.10.2 Debugging	38
2.10.2.1 Common Debugging Techniques.....	39
Chapter Summary — The Big Picture	41
End-of-Chapter Exercises	42
Multiple Choice Questions.....	42
Short Questions.....	43
Long Questions	43

Unit 2: Programming and Problem Solving with Python

An 11th Class Computer Science Study Guide

Introduction

Hey there, future programmer. Welcome to Unit 2.

Here's a secret: every single professional software developer you've ever heard of — the people who built Instagram, the people who wrote the code for Mars rovers, the people who made your favorite mobile game — started exactly where you are right now. Zero experience. A blank screen. A little bit of nervousness.

That's completely normal. In fact, that nervousness is a *good sign*. It means you're about to learn something real.

In this unit, we are going to learn **Python**, one of the most popular, most readable, and most beginner-friendly programming languages in the world. We will start from the very beginning — what a variable even *is* — and build all the way up to writing our own classes, handling errors gracefully, and reading and writing files.

Think of this chapter as your training ground. We are not just going to *read* about code. We are going to *think* like programmers: breaking big problems into small steps, testing our ideas, and — yes — making mistakes and fixing them. That last part, fixing mistakes, is not a side effect of programming. It **is** programming.

So take a breath, open your mind, and let's write some code.

2.1 Introduction to Python Programming

2.1.1 Understanding Basic Programming Concepts

2.1.1.1 Programming Basics

The Hook (Story Mode):

It's Christmas week, 1989, in the Netherlands. While most people are relaxing at home, a computer scientist named **Guido van Rossum** decides to spend his holiday building a brand-new programming language. He wants something different from the tools he had used before — something that is easy to *read*, almost like plain English, and genuinely fun to write. He is a fan of the British comedy show *Monty Python's Flying Circus*, so he names his creation **Python**.

More than three decades later, that holiday project now powers websites like Instagram and YouTube, controls robots on Mars, and trains the artificial intelligence models changing our world. All of it started with one person deciding that code should be simple enough for humans to read comfortably.

That is the spirit we carry into this whole unit: code should be clear, not clever for the sake of being clever.

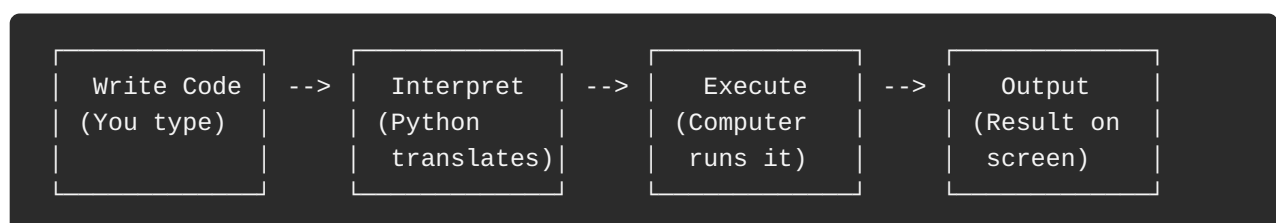
The Explanation:

Let's define our very first term. **Computer programming** is the process of giving a computer a precise set of instructions so it can complete a task. That's it. Nothing magical. You are simply writing down, very exactly, what you want the computer to do — because unlike a human, a computer will not "guess" what you meant.

Every computer program, no matter how complicated, is built using the same four basic steps:

1. **Write Code** — You type instructions in a programming language (in our case, Python) that expresses your idea.
2. **Compile/Interpret** — The computer translates your human-readable code into a form it can actually execute. Python uses an **interpreter**, a program that reads your Python code line-by-line and runs it immediately.
3. **Execute** — The computer actually carries out your instructions, one after another.
4. **Output** — The computer shows you a result: text on the screen, a saved file, a calculation, a game move — whatever your program was designed to produce.

Here is that cycle as a simple diagram:



A useful way to link this to your own life: think about following a recipe to bake a cake. You *write* the recipe steps, your brain *interprets* what each instruction means, your hands *execute* the mixing and baking, and the *output* is the cake. Programming is exactly this same logic, just applied to a computer instead of an oven.

The Practical Walkthrough:

There's no code to run yet in this section, but let's mentally walk through the four-step cycle with a real example:

1. **Write Code:** Imagine you type `print("Hello, world!")`.
2. **Interpret:** Python reads this line and understands: "Ah, the student wants to display the text `Hello, world!` on the screen."
3. **Execute:** Python runs the instruction.
4. **Output:** The words `Hello, world!` appear on your screen.

What just happened? You just traced the exact same four-step journey that every single line of code you will ever write takes — from your keyboard to the screen.

Interactive Stop-Point (Pause & Think):

Think about brushing your teeth in the morning. Break this everyday activity into four to six precise steps, written exactly like instructions for someone who has *never* brushed their teeth before (and who takes every instruction 100% literally). What happens if you forget a step, like "put toothpaste on the brush"? How is this similar to what happens when a programmer forgets a line of code?

Quick Recap:

Programming means writing precise instructions that a computer interprets, executes, and turns into visible output — nothing more mysterious than that.

2.1.1.2 Setting Up Python Development Environment

The Hook (Story Mode):

Every carpenter needs a workshop before they can build furniture. Every chef needs a kitchen before they can cook a meal. In the same way, every programmer needs a **development environment** — a workspace on the computer — before they can write and run code. Let's build yours.

The Explanation:

A **development environment** is the collection of software tools that let you write, run, and fix (debug) your code. At a minimum, you need two things:

1. **Python itself** — the actual interpreter software that reads and runs your code. You can download it for free from <https://www.python.org/>.
2. **An IDE (Integrated Development Environment)** — a specialized text editor designed for writing code. It highlights your code in color, points out obvious mistakes, and lets you run your program with one click. Popular choices include IDLE (which comes bundled with Python) and Visual Studio Code (VS Code).

Tidbit: When installing Python on Windows, make sure to check the box that says **"Add Python to PATH."** This tells your computer, "Whenever someone types `python` in the command line, know where to find it." Skipping this step is one of the most common reasons beginners get confused when Python "doesn't work."

If you don't want to install anything yet, you can also use free online Python editors in your web browser to get started immediately.

The Practical Walkthrough — Installing Python and Running Your First Program:

1. **Open your web browser** and go to `https://www.python.org/downloads/`.
2. **Click the "Download Python" button.** This downloads the installer file for your operating system.
3. **Run the installer.** A setup window will appear.
4. **Check the box "Add Python to PATH"** at the bottom of the installer window. This is the step everyone forgets — don't be everyone!
5. **Click "Install Now"** and wait for the installation to finish.
6. **Open your IDE** (IDLE, which installs automatically with Python, or VS Code if you installed it separately).
7. **Create a new file** and name it `hello.py`. The `.py` extension tells the computer, "This is a Python file."
8. **Type this single line of code:**

```
python  
print("This is my first page")
```
9. **Run the program** (in IDLE, press F5; in VS Code, click the "Run" triangle button).

What just happened? A small black window (called the **console** or **terminal**) should have popped up and displayed the text: `This is my first page`. Congratulations — you are now, officially, a programmer. You gave the computer an instruction, and it obeyed.

Interactive Stop-Point (Grab a Partner):

With a partner, each of you installs Python (or opens an online Python editor) and writes a `print()` statement that displays your own name and your favorite hobby in a single line, like `print("My name is Ahmad and I love football.")`. Run each other's code on your own machines. Did it work the first time? If not, read the error message out loud to each other — what is Python telling you?

Quick Recap:

A development environment (Python + an IDE) is your programming workshop. Once it's installed correctly — especially with "Add Python to PATH" checked — you're ready to write and run real code.

2.2 Basic Python Syntax and Structure

The Hook (Story Mode):

Every language has grammar rules — the specific order of words that makes a sentence make sense. English has grammar. Urdu has grammar. Python has grammar too, and we call it **syntax**. Get the syntax wrong, and Python won't understand you, exactly like how mixing up word order in a sentence can confuse a listener.

The Explanation:

Let's look at the simplest possible Python program:

```
print("This is my first page")
```

Here, `print` is a **function** — a built-in command that displays text on the screen. The text inside the double quotation marks `"..."` is called a **string**, which is just a fancy word for "a piece of text." The parentheses `()` tell Python, "Here is the information I want to give to the `print` function."

Comments are notes in your code that Python completely ignores when running the program. They exist purely to help *humans* — you, your teacher, or another programmer — understand what the code is doing. In Python:

- A **single-line comment** starts with the `#` symbol.
- A **multi-line comment** is wrapped in triple quotation marks `"""` at the start and end.

```
# This is a single-line comment
print("K2 is the second-highest mountain in the world")

"""
This is a multi-line comment.
It can span multiple lines.
"""
print("Edhi Foundation is the largest volunteer ambulance network.")
```

The Practical Walkthrough:

1. Open your `hello.py` file (or create a new one called `comments.py`).
2. Type the two `print()` statements above, along with their comments.
3. Run the file.

What just happened? Notice that only the text inside the `print()` statements appeared on screen. The comment lines were completely skipped by Python — they exist only for human eyes.

Interactive Stop-Point (Pause & Think):

Why would a professional programmer bother writing comments if the computer ignores them anyway? Think about a program with 500 lines of code, written six months ago by someone else. Would comments help or hurt you as the person reading it today?

Quick Recap:

Python syntax is the exact grammar Python expects; functions like `print()` perform actions, strings hold text, and comments (`#` or `"""..."""`) are notes for humans that Python simply skips over.

2.2.1 Variables, Data Types and Input/Output

2.2.1.1 Variable

The Hook (Story Mode):

Picture your bedroom. You have labeled plastic storage boxes — one says "Books," one says "Chargers," one says "Old Photos." The label never changes, but what's *inside* the box can. Today, the "Books" box might hold five novels; next month, you might empty it out and put board games in there instead. The label stays the same; the contents change.

This is exactly what a **variable** is in programming.

The Explanation:

A **variable** is a named storage location in the computer's memory. It lets you store a value, retrieve it later, and change it whenever you need to. The variable name is the "label on the box," and the value is "what's inside the box right now."

```
age = 71
print("Ahmad lived for", age, "years")

age = 60
print("Iqbal lived for", age, "years")
```

Output:

```
Ahmad lived for 71 years
Iqbal lived for 60 years
```

Notice something important: the *same* variable, `age`, held the value `71` and then later held the value `60`. The box's label didn't change — only its contents did. This is why we say a variable's value can change throughout the execution of a program.

Here's a memory map to visualize it:


```
Step 1:      age  [ 71 ]

Step 2:      age  [ 60 ]  <-- old value (71) is gone, replaced
```

The Practical Walkthrough:

1. Create a new file `variables.py`.
2. Type the code above exactly.
3. Run it and observe the two lines of output.
4. Now, add a third line: reassign `age = 45` and print another sentence using it.

What just happened? Each time you used `=` to assign a new value, Python erased the old value from that variable's memory box and stored the new one in its place.

Interactive Stop-Point (Pause & Think):

If you wrote `age = 71` and then, later in your code, wrote `Age = 60` (with a capital A), would `print(age)` show `71` or `60`? Why do you think that is?

Quick Recap:

A variable is a labeled storage box in memory: the name stays fixed, but the value inside can be created, retrieved, and changed at any time using `=`.

2.2.1.2 Variable Naming Rules in Python

The Hook (Story Mode):

Imagine trying to label a storage box with a smiley-face sticker instead of a written word. It would be confusing — nobody, not even you, could tell what's inside just by looking at the label. Python is just as strict (and helpful!) about how you name your variables, precisely so that your code stays readable.

The Explanation:

Python enforces a few clear rules for variable names:

- The name **must begin** with a letter (`a–z`, `A–Z`) or an underscore (`_`).
- **Subsequent characters** can be letters, digits (`0–9`), or underscores.
- Variable names are **case-sensitive** — `age` and `Age` are treated as two completely different variables.

- You **cannot** use Python's reserved keywords (like `for`, `while`, `if`) as variable names, because Python has already reserved those words for its own grammar.

Tidbit: Always use meaningful names. Prefer `age` over a single meaningless letter like `a` — your future self (and your teammates) will thank you.

The Practical Walkthrough:

1. Open a new file `naming_rules.py`.
2. Try typing each of these and predict, before running, whether Python will accept it:

```
python
student_name = "Ali"
_score = 95
2nd_place = "Sara"      # try this one – what happens?
class = "11th"          # try this one too – what happens?
```

3. Run the file and read any error message carefully.

What just happened? Lines 1 and 2 run without any problem. Lines 3 and 4 cause a **SyntaxError** — Python's way of saying, "This name breaks my grammar rules." `2nd_place` starts with a digit, and `class` is a reserved keyword.

Interactive Stop-Point (Pause & Think):

Look at these three variable names: `2nd_student`, `student_name`, and `class`. Which ones are illegal in Python, and exactly which rule does each one break?

Quick Recap:

Variable names must start with a letter or underscore, may contain letters/digits/underscores after that, are case-sensitive, and can never be a reserved Python keyword.

2.2.1.3 Creating Different Types of Variables

The Hook (Story Mode):

Not everything in your storage box is the same *kind* of item. Some boxes hold whole objects (like a stack of books), some hold liquids that can be measured to a fraction (like a half-full water bottle), some hold written notes (text), and some are simply "yes-or-no" light switches. Python organizes data the exact same way, using **data types**.

The Explanation:

A **data type** tells Python what *kind* of value a variable is holding, and therefore what operations make sense on it. Python has four data types you'll use constantly at this stage:

Data Type	Keyword	Stores	Example
Integer	<code>int</code>	Whole numbers	<code>age = 17</code>
Floating-point	<code>float</code>	Decimal numbers	<code>price = 19.99</code>
String	<code>str</code>	Text	<code>name = "Ali"</code>
Boolean	<code>bool</code>	<code>True</code> or <code>False</code>	<code>is_student = True</code>

Tidbit: It's good practice to use lowercase letters for variable names, with underscores separating words — for example, `student_name` rather than `StudentName` or `studentname`.

The Practical Walkthrough:

1. Create `data_types.py`.
2. Create one variable of each type from the table above.
3. Add `print(type(age))`, `print(type(price))`, `print(type(name))`, and `print(type(is_student))` to your file.
4. Run the program.

What just happened? Python's built-in `type()` function reveals the data type Python assigned automatically. You'll see `<class 'int'>`, `<class 'float'>`, `<class 'str'>`, and `<class 'bool'>`. Python figures out the type on its own, just from how you wrote the value — no digits and quotes means a number; quotes mean text.

Interactive Stop-Point (Pause & Think):

Is `"25"` (with quotation marks) the same as `25` (without quotation marks) to Python? What data type is each one? Why might this difference cause a problem if you tried to add them together?

Quick Recap:

Python has four essential data types at this stage — `int`, `float`, `str`, and `bool` — and Python automatically detects which one you mean based on how you write the value.

2.2.1.4 Input and Output Operations

The Hook (Story Mode):

Think about an ATM machine. It doesn't just show you a fixed message — it *asks* you something ("Enter your PIN") and *waits* for your response before doing anything else. Programs need this same two-way conversation ability: asking for information (**input**) and showing results (**output**).

The Explanation:

- **Input:** The `input()` function displays a message (called a **prompt**) on the screen, then pauses the program and waits for the user to type something and press Enter. Whatever the user types is stored, as a string, in a variable.

```
python
```

```
name = input("Enter your name: ")
```

- **Output:** The `print()` function displays information on the screen. You can combine text and variables using the `+` operator (this is called **string concatenation**).

```
python
```

```
print("Hello, " + name + "!")
```

The Practical Walkthrough:

1. Create `greeting.py`.
2. Type both lines of code above.
3. Run the program. Your console will pause and show `Enter your name:` — type your name and press Enter.

What just happened? Your program paused execution at the `input()` line, waited for you (the human) to respond, stored your typed text in the `name` variable, and then used it inside the `print()` statement.

Interactive Stop-Point (Pause & Think):

What data type does `input()` *always* return, no matter what the user types — even if they type a number like `25`? What problems could this cause if you tried to do math with it directly?

Quick Recap:

`input()` pauses your program to collect text from the user (always as a string), and `print()` displays output — together they let your program hold a two-way "conversation" with a human.

2.2.1.5 Handling Integer and Float Inputs

The Hook (Story Mode):

Remember the "Pause & Think" question above? You caught the problem yourself: `input()` always hands back text, even if the user typed digits. If you tried to do math with that text directly, Python would get confused, the same way you might get confused if someone handed you the *word* "five" instead of the *number* 5 and asked you to add it to 3.

The Explanation:

To fix this, we **convert** the text returned by `input()` into a real number using two built-in functions:

- `int()` — converts text into a whole number.
- `float()` — converts text into a decimal number.

```
# Integer input
user_age = int(input("Enter your age: "))
print("Your age is:", user_age)

# Float input
user_height = float(input("Enter your height in meters: "))
print("Your height is", user_height, "meters")
```

The Practical Walkthrough:

1. Create `numeric_input.py`.
2. Type both examples above.
3. Run the program and enter `17` when asked for your age, then `1.72` when asked for your height.
4. Now try adding `user_age + 5` and printing the result, to prove it's now truly numeric.

What just happened? By wrapping `input()` inside `int()` or `float()`, you converted the text the user typed into a genuine number that Python can now use in mathematical calculations.

Interactive Stop-Point (Grab a Partner):

Partner A writes a small program that asks for a user's height in meters using `float(input(...))`. Partner B intentionally types a non-numeric answer, like `"tall"`, when running it. What happens? Read the error message together — what is Python's `ValueError` trying to tell you?

Quick Recap:

Because `input()` always returns text, wrap it in `int()` or `float()` whenever you need to perform real math on what the user typed.

2.3 Operators and Expressions

The Hook (Story Mode):

Think of a calculator app on your phone. The `+`, `-`, `×`, and `÷` buttons are **operators** — symbols that perform an action on numbers. Python has the exact same idea, just written slightly differently in code, plus several more operator types for comparing and combining values.

The Explanation:

An **operator** is a symbol that performs an operation on one or more values. An **expression** is any combination of variables, operators, and values that produces a result — for example, `3 + 4` is an expression that evaluates to `7`.

2.3.1 Arithmetic Operators

Arithmetic operators perform basic math:

```
a, b = 10, 3

print(a, "+", b, "=", a + b)    # Addition      -> 13
print(a, "*", b, "=", a * b)    # Multiplication -> 30
print(a, "/", b, "=", a / b)    # Division      -> 3.3333333333333335
print(a, "//", b, "=", a // b)  # Floor Division -> 3
print(a, "%", b, "=", a % b)    # Modulus       -> 1
print(a, "**", b, "=", a ** b)   # Exponentiation -> 1000
```

DO YOU KNOW? A full official tutorial on Python is freely available at <https://docs.python.org/3/tutorial/>.

2.3.2 Comparison Operators

Comparison operators compare two values and always return a **Boolean** result — `True` or `False`.

```
x, y = 10, 5

print(x, ">", y, "=", x > y)    # True
print(x, "<", y, "=", x < y)    # False
print(x, "==", y, "=", x == y) # False (equality check, NOT assignment!)
print(x, "!=", y, "=", x != y) # True
print(x, ">=", y, "=", x >= y)  # True
print(x, "<=", y, "=", x <= y)  # False
```

Crucial distinction: `=` assigns a value; `==` checks whether two values are equal. Confusing these two is one of the most common beginner bugs in all of programming.

2.3.3 Assignment Operators

The `=` sign assigns a value to a variable. Python also offers **compound assignment operators** that combine an arithmetic operation with assignment in one step:

```
a, b = 10, 5

a += b    # a after addition      = 15
a -= b    # a after subtraction   = 10
a *= b    # a after multiplication = 50
a /= b    # a after division      = 10.0
a %= b    # a after modulus       = 0.0
a **= b   # a after exponentiation = 0.0
```

`a += b` is simply a shorter way of writing `a = a + b`.

2.3.4 Logical Operators

Logical operators combine multiple True/False conditions: `and`, `or`, and `not`.

```
x = True
y = False

print(x, "and", y, "=", x and y)  # False (BOTH must be True)
print(x, "or", y, "=", x or y)    # True  (at least ONE must be True)
print("not x =", not x)           # False (flips the value)
```

2.3.5 Expressions

An expression combines variables, operators, and values into a single result:

```
result = (3 + 4) * 2    # result is 14
```

Parentheses `()` control the order in which parts of the expression are calculated.

Interactive Stop-Point (Class Activity — Pause & Think):

Write a program to calculate **Body Mass Index (BMI)**. Ask the user for their weight (in kilograms) and height (in meters), then compute:

```
BMI = weight / (height ** 2)
```

Print the resulting BMI. What data type conversions (`float()`) will you need for the inputs?

2.3.6 Operator Precedence in Python

The Hook (Story Mode):

You learned "PEMDAS" or "BODMAS" in math class — Parentheses, Exponents, Multiplication/Division, Addition/Subtraction. Python follows almost exactly this same order when it evaluates an expression.

The Explanation:

Operator precedence determines *which* operation Python performs first in an expression with multiple operators:

1. **Parentheses** `()` — highest precedence, evaluated first. `(3 + 2) * 4` → `20`
2. **Exponentiation** `**` — next. `2 ** 3` → `8`
3. **Multiplication** `*`, **Division** `/`, **Modulus** `%` — next, left to right. `4 * 3` → `12`, `10 / 2` → `5.0`, `11 % 3` → `2`
4. **Addition** `+` and **Subtraction** `-` — lowest precedence among these. `5 + 2` → `7`, `10 - 4` → `6`

DO YOU KNOW? Using extra parentheses — even when not strictly required — is a great habit. It makes complex expressions easier for *humans* to read correctly, even when Python would technically compute the same answer without them.

Interactive Stop-Point (Class Activity — Grab a Partner):

Compute these two expressions by hand first, then verify with real Python code, and compare your manual answer with a partner's:

1. `10 + 3 * 2 ** 2 - 5 / 5`
2. `(10 + 3) * (2 ** (2 - 1)) / 5`

Quick Recap:

Operators let you perform math (`arithmetic`), comparisons (`comparison`), assignment (`assignment`), and logic (`logical`) on values; Python evaluates expressions using a strict precedence order — parentheses first, then exponents, then multiplication/division, then addition/subtraction.

2.4 Control Structures

The Hook (Story Mode):

So far, every program we've written runs top to bottom, one line after another, no matter what. But real life isn't like that. Should you carry an umbrella? *It depends* on whether it's raining. Should you keep studying? *It depends* on whether the exam is tomorrow. **Control structures** give our programs this same ability to make decisions and repeat actions — exactly the way you make decisions every day.

The Explanation:

There are two main families of control structures in programming:

- **Decision Making** — choosing different actions based on a condition (if/else logic).
- **Looping** — repeating an action multiple times (while/for logic).

2.4.1 Decision Making

2.4.1.1 If Statement

The Hook: Imagine a simple rule: "If it's raining, take an umbrella." Nothing happens if it's *not* raining — you simply skip the umbrella step.

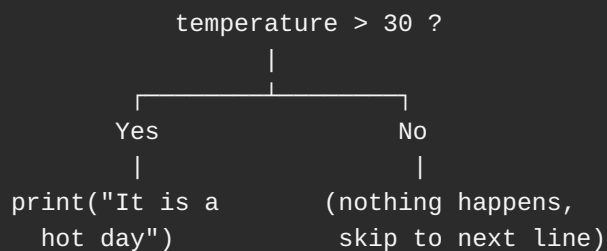
The Explanation:

```
if condition:
    # code to run if the condition is true
```

Example:

```
temperature = 35
if temperature > 30:
    print("It is a hot day")
```

Notice the **colon** `:` at the end of the `if` line, and the **indentation** (the space before `print`). In Python, indentation is not just style — it's *syntax*. It tells Python exactly which lines belong "inside" the `if` block.



Interactive Stop-Point (Pause & Think):

If `temperature = 25`, will the `print()` line run in the example above? Trace through the logic yourself before checking by running the code.

2.4.1.2 If-else Statement

The Explanation:

```
if condition:
    # code to run if condition is true
else:
    # code to run if condition is false
```

Example:

```
temperature = 15
if temperature > 30:
    print("It's a hot day")
else:
    print("It's not a hot day")
```

Now every path leads somewhere — either the `if` block runs, or the `else` block runs. Exactly one of the two always executes.

Interactive Stop-Point (Grab a Partner):

Partner A picks a secret temperature value and doesn't tell Partner B. Partner B asks yes/no-style questions to guess which branch (`if` or `else`) would run. Then reveal the value and check the logic together.

2.4.1.3 Short Hand if-else Statement

The Explanation:

Python allows writing a simple if-else in a single line, called a **conditional expression** or **ternary expression**:

```
action_if_true if condition else action_if_false
```

Example:

```
temperature = 15
m = "It's a hot day" if (temperature > 30) else "It's not a hot day"
print(m)
```

This does exactly the same thing as the full `if-else` block from the previous section — it's just more compact, useful for short, simple decisions.

Interactive Stop-Point (Class Activity — Pause & Think):

Write both a regular `if-else` statement *and* a short-hand `if-else` statement that check whether a number is even or odd (hint: use the modulus operator `%` — a number is even if `number % 2 == 0`). Which version do you find easier to read?

2.4.1.4 if-elif-else Statement

The Hook (Story Mode):

Real decisions are rarely just two options. Think about checking the weather forecast: sunny, rainy, or cloudy — three (or more!) possible outcomes, each needing a different response.

The Explanation:

```
if condition1:
    # code to run if condition1 is true
elif condition2:
    # code to run if condition2 is true
else:
    # code to run if none of the conditions are true
```

Example:

```
weather = "cloudy"

if weather == "sunny":
    print("Wear sunglasses")
elif weather == "rainy":
    print("Take an umbrella")
else:
    print("Enjoy your day!")
```

Python checks each condition **in order**, from top to bottom. The moment one condition is **True**, that block runs, and Python **skips all the rest** — even if a later condition would also have been **True**.

Interactive Stop-Point (Class Activity — Grab a Partner):

Write an **if-elif-else** statement that checks whether a number is positive, negative, or zero. Partner A writes the code; Partner B tests it with three different numbers (one positive, one negative, one zero) and confirms each output is correct.

Quick Recap (2.4.1):

if, **if-else**, short-hand **if-else**, and **if-elif-else** all let your program choose different paths of execution based on conditions — from a single yes/no branch up to many possible branches.

2.4.2 Looping Constructs

2.4.2.1 while Loop

The Hook (Story Mode):

In 1843, mathematician **Ada Lovelace** wrote what is widely considered the world's first computer algorithm — a set of instructions for Charles Babbage's mechanical **Analytical Engine** to calculate a sequence of numbers called Bernoulli numbers. Her algorithm relied on *repeating* a set of steps over and over, checking a condition each time to know when to stop. That is precisely what a **loop** does, over 180 years later, inside every computer on Earth.

The Explanation:

A `while` loop repeats a block of code **as long as** a condition remains `True`. It checks the condition *before* every single repetition (called an **iteration**), and stops the moment the condition becomes `False`.

```
# Syntax
while condition:
    # code to run while the condition is true
```

Example:

```
number = 1
while number < 10:
    print(number)
    number += 1
```

Execution trace table:

Iteration	<code>number</code> value	<code>number < 10</code> ?	Action
1	1	True	print 1, then number becomes 2
2	2	True	print 2, then number becomes 3
...	...	True	... continues ...
9	9	True	print 9, then number becomes 10
10	10	False	loop stops

Critical warning: If you forget the line `number += 1`, the condition `number < 10` would *never* become false, and your program would run forever. This is called an **infinite loop** — always double-check that something inside your loop actually moves you toward the stopping condition.

Interactive Stop-Point (Class Activity — Pause & Think):

Write a Python program using a `while` loop that prints only the **even** numbers and separately counts the **odd** numbers from 1 to 20. What condition will you check inside the loop to tell even from odd?

2.4.2.2 for Loop

The Hook (Story Mode):

Imagine you're greeting every single friend at a birthday party, one by one, going around the room. You don't need to count how many friends there are in advance — you simply visit each one in turn until you've greeted everybody. This is exactly the mental model behind a Python `for` loop.

The Explanation:

A `for` loop repeats a block of code once **for each item** in a sequence (like a list, tuple, or string) — no manual counting variable required.

```
# Syntax
for variable in sequence:
    # code to run for each element in the sequence
```

Example:

```
friends = ["Ahmad", "Ali", "Hassan"]

for friend in friends:
    print("Hello", friend)
```

Output:

```
Hello Ahmad
Hello Ali
Hello Hassan
```

Explanation: The loop goes through each `friend` in the `friends` list, one at a time, printing a greeting for each one — automatically stopping once it reaches the end of the list.

The `range()` function is often used with `for` loops to repeat something a specific number of times:

```
for i in range(1, 6):    # generates 1, 2, 3, 4, 5
    print(i)
```

The Practical Walkthrough:

1. Create `loops.py`.
2. Type the `friends` list example above and run it.
3. Add a second `for` loop using `range()` that prints numbers 1 through 5.
4. Run again and compare the two outputs.

What just happened? You saw two different ways to use a `for` loop: iterating directly over a list of real values, and iterating over a generated sequence of numbers from `range()`.

Interactive Stop-Point (Class Activity — Grab a Partner):

1. Write a `for` loop using `range()` to print the even numbers from 2 to 10.
2. Write a Python program that prints the first 10 multiples of 3 using a `for` loop and `range()`.

Compare your solutions with a partner — did you both use the same `range()` arguments?

Quick Recap (2.4.2):

A `while` loop repeats based on a condition being checked before every iteration (useful when you don't know exactly how many repetitions you need), and a `for` loop repeats once per item in a sequence (useful when iterating over a known collection).

2.5 Python Modules and Built-in Data Structures

The Explanation:

Python comes with an enormous **standard library** — a huge toolbox of pre-written, ready-to-use code, organized into **modules**. A **data structure** is simply a particular way of organizing and storing data (a list, which we'll explore soon, is one such data structure). In this section, we look at how to write your own reusable code (functions), and how to borrow code that others have already written (libraries and modules).

2.5.1 Functions and Modules

The Hook (Story Mode):

Think of a vending machine. You put in coins and press a button (your **input**), an internal mechanism does its work out of sight (the **process**), and a snack drops out (the **output**). You never need to know exactly how the machine's gears work inside — you just trust the result. A **function** in programming works exactly the same way.

The Explanation:

A **function** is a named, reusable block of code that performs a specific task. Instead of retyping the same instructions over and over, you define them once and *call* (invoke) them by name whenever you need them.

2.5.1.1 Defining and Invoking Functions

```
def function_name(parameters):  
    # code to be executed
```

Example:

```
def greet(name):  
    print("Hello", name)  
  
greet('Ali')
```

Defining a function (using `def`) is like writing the recipe. **Invoking** (or calling) a function — like `greet('Ali')` — is like actually following that recipe with a specific set of ingredients.

2.5.1.2 Function Parameters and Return Values

Functions can accept multiple **parameters** (the inputs) and hand back a **return value** (the output) using the `return` keyword.

```
def add(a, b):  
    return a + b
```

DO YOU KNOW? You can call the same function many times with completely different arguments, reusing the exact same code for different inputs — that's the entire point of writing a function in the first place.

Interactive Stop-Point (Class Activity — Pause & Think):

Define a function that takes a list of numbers as its parameter and returns the maximum value in that list. (Hint: Python has a built-in `max()` function — but try writing the logic yourself using a loop first, to understand what's happening underneath.)

2.5.1.3 Default Parameters

Functions can have **default parameter values**, which are automatically used if the caller doesn't provide their own argument.

```
def greet(name="Student"):  
    return "Hello, " + name + "!"  
  
print(greet())           # Output: Hello, Student!  
print(greet("Umer"))     # Output: Hello, Umer!
```

The Practical Walkthrough:

1. Create `functions.py`.
2. Write the `add(a, b)` function and call it with two different pairs of numbers, printing each result.
3. Write the `greet(name="Student")` function with a default parameter and call it once with no argument, once with an argument.
4. Run and observe both behaviors.

What just happened? You saw that a function's behavior changes based on the arguments you provide — and that a default parameter lets your function still work sensibly even when no argument is given.

2.5.2 Using Libraries and Modules

In Python, **libraries** and **modules** are like ready-made toolboxes, packed with tools that solve common problems so you don't have to build everything from scratch yourself.

2.5.3 Importing and Using Libraries

You bring a library's tools into your program using the `import` keyword.

```
import random

# Generate a random number between 1 and 10
number = random.randint(1, 10)
print("The random number is:", number)
```

```
import datetime

# Get the current date and time
current_time = datetime.datetime.now()
print("Current date and time:", current_time)
```

```
import statistics

# Calculate the mean of a list of numbers
data = [23, 45, 67, 89, 12, 44, 56]
mean_value = statistics.mean(data)
print("The mean value is:", mean_value)
```

2.5.3.1 Package Structure

To organize large projects, related modules are grouped into a **package** — simply a directory (folder) containing related module files. For example, an e-commerce project might have a package named `ecommerce` containing modules like `products.py`, `customers.py`, and `orders.py`.

Example — inside `ecommerce/products.py`:

```
def list_products():
    return ["Laptop", "Mobile", "Tablet"]
```

In your main script:


```
from ecommerce import products

available_products = products.list_products()
print(available_products)
# Output: ['Laptop', 'Mobile', 'Tablet']
```

Tidbit: Organizing modules into packages is like organizing books into labeled sections of a library — it makes everything much easier to find and maintain.

Interactive Stop-Point (Pause & Think):

Why would a large software company (say, one building a banking app with thousands of lines of code) insist on organizing their code into packages and modules, rather than writing every single function inside one giant file?

Quick Recap (2.5):

Functions let you package reusable logic with inputs (parameters) and outputs (return values), often with sensible defaults, while modules and packages let you organize and reuse large amounts of pre-written code — your own, or the Python standard library's.

2.6 Built-in Data Structures

The Explanation:

Python provides several built-in **data structures** for organizing and manipulating collections of data efficiently: **lists**, **tuples**, and (in later units) **dictionaries**. Each has unique strengths for different situations.

2.6.1 Lists

The Hook (Story Mode):

Picture a shopping list written on a whiteboard hanging in your kitchen. You can add a new item anytime, erase something you no longer need, or change an item's name entirely. A Python **list** behaves exactly like that whiteboard — flexible and changeable.

2.6.1.1 Creating, Accessing, and Modifying Lists

A **list** is created using square brackets `[]`, with items separated by commas. Lists can hold items of different types.

```
fruits = ["Mango", "Apple", "Banana"]
print(fruits)
# Output: ['Mango', 'Apple', 'Banana']
```

2.6.1.2 Accessing List Items

You access an item by its **index** — its position number, starting from **0**, not **1**.

```
fruits = ["Mango", "Apple", "Banana"]
print(fruits[1])
# Output: Apple
```

Explanation: Index **0** is **"Mango"**, index **1** is **"Apple"**, and index **2** is **"Banana"**. This is called **zero-based indexing**, and it's one of the very first "gotchas" every new programmer runs into — so take note of it now.

```
Index:    0        1        2
fruits: ["Mango", "Apple", "Banana"]
```

2.6.1.3 Modifying a List

Because lists are **mutable** (changeable), you can update an item by index, or add new items:

```
fruits = ["Mango", "Apple", "Banana"]
fruits[0] = "Orange"
fruits.append("Pineapple")
print(fruits)
# Output: ['Orange', 'Apple', 'Banana', 'Pineapple']
```

2.6.1.4 Methods and Operations on Lists

Useful built-in list methods:

Method	What it does
<code>append(item)</code>	Adds an item to the end of the list
<code>remove(item)</code>	Removes the first occurrence of an item
<code>sort()</code>	Sorts the list in ascending order
<code>reverse()</code>	Reverses the order of the list

```
students = ["Ahmed", "Sara", "Ali"]
students.append("Hina")
students.sort()
print(students)
# Output: ['Ahmed', 'Ali', 'Hina', 'Sara']
```

2.6.1.5 List Operations

Lists support **slicing** (extracting a portion) and **concatenation** (joining two lists with **+**):

```

numbers = [1, 2, 3, 4, 5]
slice = numbers[1:4]           # items from index 1 up to (not including) 4
extra_numbers = [6, 7]
combined = slice + extra_numbers
print(combined)
# Output: [2, 3, 4, 6, 7]

```

```

student_names = ["Ahmed", "Sara", "Ali", "Hina"]
student_names.sort()
student_names.remove("Sara")
print(student_names)
# Output: ['Ahmed', 'Ali', 'Hina']

```

The Practical Walkthrough:

1. Create `lists.py`.
2. Build a list of your five favorite movies.
3. Print the whole list, then print just the third movie using its index.
4. Change the first movie in the list to a different title using index assignment.
5. Use `.append()` to add a sixth movie.
6. Use `.sort()` to alphabetize the list, then print the final result.

What just happened? You performed the full life cycle of working with a list: creating it, reading from it by index, and modifying it — proving, hands-on, that lists are mutable.

Interactive Stop-Point (Class Activity — Pause & Think):

Starting with this list of favorite books:

```

books = ["To Kill a Mockingbird", "1984", "The Great Gatsby", "Pride and Prejudice"]

```

1. Add a new book, `"Moby Dick"`, to the list.
2. Replace `"1984"` with `"Brave New World"`.
3. Remove `"The Great Gatsby"` from the list.
4. Merge this list with `["War and Peace", "Hamlet"]`.
5. Print the final list.

Tidbit: Use list methods like `append()` and `remove()` to efficiently manage and modify your lists — for larger projects, organized lists keep your code clean and manageable.

Quick Recap (2.6.1):

A list is an ordered, mutable collection created with `[]`, accessed by zero-based index, and manipulated using methods like `append()`, `remove()`, `sort()`, and `reverse()`.

2.6.2 Tuples

The Hook (Story Mode):

Now picture a printed birth certificate. Unlike a whiteboard shopping list, you cannot casually cross out and rewrite the date of birth printed on it — that data is meant to stay fixed, permanently. A Python **tuple** is built for exactly this kind of "locked" data.

The Explanation:

A **tuple** stores an ordered collection of items, just like a list, but with one crucial difference: tuples are **immutable** — once created, their values cannot be changed.

```
my_tuple = (1, 2, 3, "Hello", 4.5)

print(my_tuple[0])      # Output: 1
print(my_tuple[3])      # Output: Hello
print(len(my_tuple))    # Output: 5
```

Tuples use round parentheses `()` instead of square brackets, and `len()` tells you how many items are inside.

Interactive Stop-Point (Pause & Think):

If lists are mutable and tuples are immutable, why would a developer deliberately choose a tuple over a list to store something like a PIN code or a pair of GPS coordinates? What real-world risk does immutability protect against?

Quick Recap (2.6.2):

A tuple is an ordered, **immutable** collection — perfect for data that should never accidentally change once it's created.

2.6.3 Indexing and Slicing

The Explanation:

Indexing and **slicing** are essential techniques for accessing and manipulating sequences (lists, tuples, and strings alike).

2.6.3.1 Indexing

Indexing accesses one specific element using its position. Python uses **zero-based indexing** — the first element is at index `0`.

2.6.3.2 Slicing

Slicing extracts a *subset* of a sequence, using the syntax:

```
sequence[start:stop:step]
```

where **start** is the first index included, **stop** is the index *before which* slicing ends (not inclusive), and **step** is how many positions to move each time.

2.6.3.3 Indexing and Slicing with Negative Indices

Negative indices count backward from the end of the sequence: **-1** is the last element, **-2** is the second-to-last, and so on.

```
fruits = ["Apple", "Banana", "Cherry", "Date", "Elderberry"]

# Indexing
print("First fruit:", fruits[0])      # Positive index
print("Last fruit:", fruits[-1])     # Negative index

# Slicing with positive indices
print("Fruits from index 1 to 3:", fruits[1:4])

# Slicing with negative indices
print("Fruits from index -4 to -1:", fruits[-4:-1])
```

Visual map of the same list, showing both positive and negative indices:

Value:	Apple	Banana	Cherry	Date	Elderberry
Positive:	0	1	2	3	4
Negative:	-5	-4	-3	-2	-1

The Practical Walkthrough:

1. Create `indexing_slicing.py`.
2. Recreate the `fruits` list above.
3. Print the first and last fruit using positive and negative indexing respectively.
4. Print a slice containing the middle three fruits.
5. Experiment: change the **step** value in a slice, e.g., `fruits[::-2]`, and observe the result.

What just happened? You practiced reading both ends of a sequence — from the front using positive indices, and from the back using negative indices — plus extracting custom "chunks" using slicing.

Interactive Stop-Point (Class Activity — Grab a Partner):

Given:

```
my_list = [10, 20, 30, 40, 50, 60, 70, 80]
my_tuple = ("Math", "Science", "English", "History", "Geography")
my_string = "Python Programming"
```

1. Access and print the third element from each sequence.
2. Slice and print elements from index 2 to 5 from the list and the tuple.
3. Slice and print characters from index 7 to the end of the string.
4. Use negative indexing to print the last two elements from the list and the tuple.
5. Use negative slicing to print the last two characters of the string.

Tidbit: Indexing and slicing are powerful tools you'll use constantly — the more you practice with real sequences, the more natural zero-based and negative indexing will feel.

Quick Recap (2.6.3):

Indexing accesses a single element by position (positive from the front, negative from the back), while slicing (`start:stop:step`) extracts an entire sub-sequence at once.

2.7 Modular Programming in Python

The Hook (Story Mode):

Imagine trying to build an entire car engine as a single, giant, inseparable block of metal. Impossible to repair, impossible to upgrade one part without rebuilding everything. Real engines are built from separate, well-defined components — pistons, spark plugs, the fuel pump — each doing one job, each replaceable on its own. **Modular programming** applies this same engineering wisdom to code.

The Explanation:

Modular programming means dividing a program into smaller, manageable, reusable pieces called **modules**. This lets different developers work on different parts independently, and lets code be reused instead of rewritten.

The Main Function

The `main()` function defines where your program should logically "start." It's typically placed inside a check that verifies whether the script is being run directly, or simply imported by another file:

```
# main.py
def main():
    print("This is the main function.")

if __name__ == "__main__":
    main()
```

Explanation: The `main()` function only runs automatically when this specific file is executed *directly*. If another file imports this one (to reuse a function from it, for example), `main()` will **not** run automatically — a very useful safety feature in larger, multi-module projects.

Tidbit: Using a `main()` function alongside well-organized modules keeps your code easy to navigate and maintain, especially as your projects grow beyond a single file.

DO YOU KNOW? Python's standard library contains hundreds of ready-made modules for common tasks — working with dates, generating random numbers, reading files, and much more.

The Practical Walkthrough:

1. Create a new folder for this exercise.
2. Inside it, create `calculator.py` containing two functions:

```
```python
def add(a, b):
 return a + b
```

```
def subtract(a, b):
 return a - b
```

3. In the same folder, create `main.py`: `python`  
`import calculator`

```
print(calculator.add(15, 8))
print(calculator.subtract(25, 10))
```

```
```
```

4. Run `main.py` (not `calculator.py`).

What just happened? You split your logic into two files: `calculator.py` holds reusable functions, and `main.py` imports and uses them. This is modular programming in its simplest, most practical form.

Interactive Stop-Point (Class Activity — Grab a Partner):

Following the walkthrough above, confirm with a partner that `main.py` correctly prints:

```
23
15
```

Now, try running `calculator.py` directly by itself — does anything print? Discuss why or why not.

Quick Recap:

Modular programming splits large programs into smaller, independent, reusable files (modules); the `if __name__ == "__main__":` pattern ensures a file's "main" logic only runs when that file is executed directly.

2.8 Object-Oriented Programming in Python

The Hook (Story Mode):

In 1962, two Norwegian computer scientists, **Ole-Johan Dahl** and **Kristen Nygaard**, were working on simulations — modeling real-world things like ships moving through a harbor. They realized it would be far more natural to represent each ship in code as its own self-contained "thing," bundling together its own data (speed, position, cargo) and its own behaviors (move, dock, load cargo). Their language, **Simula**, became the ancestor of every **Object-Oriented Programming (OOP)** language used today, including Python.

The Explanation:

Object-Oriented Programming is a way of designing and organizing code around real-world "things" (**objects**), making complex systems easier to reason about, manage, and extend.

2.8.1 Class and Objects

Think of building a toy car. Before you can build any actual toy car, you need a **blueprint** describing what every toy car of this design should have: a color, a size, a number of wheels, and the material it's built from. That blueprint is not itself a toy car — it's just a *plan*. In programming, that blueprint is called a **class**. Every actual toy car you build from the blueprint is called an **object** (or an **instance**) of that class.

2.8.1.1 Defining Classes and Creating Objects

```
class ToyCar:
    # The __init__ method initializes the object with specific attributes
    def __init__(self, color, size, wheels):
        self.color = color    # Color of the toy car
        self.size = size      # Size of the toy car
        self.wheels = wheels  # Number of wheels on the toy car

    # Method to describe the toy car
    def describe(self):
        return f"This toy car is {self.color}, size {self.size}, and has {self.wheels} wheels."

# Create objects of the ToyCar class
car1 = ToyCar("red", "small", 4)
car2 = ToyCar("blue", "large", 6)

# Print descriptions of the toy cars
print(car1.describe())
print(car2.describe())
```

Output:

```
This toy car is red, size small, and has 4 wheels.
This toy car is blue, size large, and has 6 wheels.
```

Explanation, term by term:

- **Class Definition:** `ToyCar` is the blueprint. It describes what attributes every toy car object should have: `color`, `size`, and `wheels`.
- **`__init__` method:** This is a special method that runs automatically the moment a new object is created, setting up its initial attribute values. Think of it as the "assembly line" that builds each new toy car according to the blueprint.
- **Creating Objects:** `car1` and `car2` are actual toy car objects, each built from the `ToyCar` blueprint, but each holding its own unique attribute values.
- **Using Methods:** `describe()` is a **method** — a function that belongs to the class — that lets each object describe itself.
- **`self`:** By convention, `self` represents "this specific object" inside a class's methods. When you call `car1.describe()`, Python understands `self` to mean `car1` specifically, not `car2`.

The Practical Walkthrough:

1. Create `toy_car.py`.
2. Type the full `ToyCar` class exactly as shown above.

3. Create a third object, `car3`, with your own choice of color, size, and wheel count.
4. Print `car3.describe()`.

What just happened? You created a brand-new object from the exact same blueprint, proving that a single class can produce as many independent objects as you need, each with its own data.

Interactive Stop-Point (Pause & Think):

If you changed `car1.color = "green"` after creating the object, would `car2.color` also change? Why or why not? What does this tell you about how independent each object is from the others, even though they share the same class?

Quick Recap:

A class is a blueprint describing shared structure (attributes) and behavior (methods); an object is one specific instance created from that blueprint, with its own independent data — and `self` always refers to the specific object currently being worked with.

2.9 Advanced Python Concepts

The Explanation:

Now that you have a solid foundation, let's tackle two skills that separate hobby code from *robust, real-world-ready* code: gracefully handling errors, and saving/loading data using files.

2.9.1 Exception Handling

The Hook (Story Mode):

No responsible driver plans to get a flat tire. But every car still carries a spare tire in the trunk, just in case. When a tire bursts unexpectedly (an **exception**), the driver doesn't abandon the trip — they calmly pull over, swap in the spare, and continue safely. **Exception handling** in Python is your program's spare tire.

The Explanation:

An **exception** is an error that occurs while your program is running. Left unhandled, it crashes your program immediately. **Exception handling** lets your program detect the error and respond gracefully instead of crashing.

2.9.1.1 Try-Except Blocks

```
a = int(input("Enter a number: "))
try:
    result = 10 / a    # This line raises an error if a is 0
except ZeroDivisionError:
    print("You can't divide by zero!")
```

Explanation:

- The **try** block contains code that *might* cause an error.
- The **except** block catches a specific type of error (here, **ZeroDivisionError**) and handles it — in this case, by printing a friendly message instead of letting the program crash.

Reframing errors: An error message is not a punishment. It is Python's precise, honest status report, telling you exactly where and why it got confused. Every professional developer, no matter how experienced, reads error messages every single day. Learning to read them calmly and carefully — like a detective reading clues — is one of the single most valuable skills you will build in this entire course.

The Practical Walkthrough:

1. Create **try_except.py**.
2. Type the code above.
3. Run it and enter **0** when prompted — observe the friendly message instead of a crash.
4. Run it again and enter a normal number like **5** — observe the calculation succeed.
5. Now run it a third time and type a letter, like **"abc"**, instead of a number. You'll get a *different* error — a **ValueError** — because your **except** block only catches **ZeroDivisionError**. Add a second **except ValueError:** block to handle this case too.

What just happened? You discovered that different problems raise different, specific exception types, and that you can catch multiple types of errors by adding more **except** blocks.

Interactive Stop-Point (Pause & Think):

A user is prompted to enter their birth year but types **"nineteen-ninety-nine"** in words instead of digits. Trace through what error Python would raise, and describe exactly how a **try-except** block could handle it gracefully instead of crashing the program.

Quick Recap (2.9.1):

try wraps risky code, and **except** catches specific errors when they happen, letting your program respond gracefully rather than crashing — errors are simply Python's honest status reports, not personal failures.

2.9.2 File Handling

The Hook (Story Mode):

Every variable you've created so far disappears the instant your program finishes running — like writing notes on a whiteboard that gets erased at the end of class. **File handling** lets your program write notes on paper instead: information that survives after the program closes, so you (or another program) can read it again later.

The Explanation:

File handling means reading from and writing to files on your computer, allowing your program to store data **persistently** (i.e., permanently, beyond a single run).

2.9.2.1 Opening, Reading, and Closing Files

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

Explanation:

- The `open()` function opens a file. The `"r"` argument means **read mode**.
- The `with` statement is a safety net: it guarantees the file is properly closed automatically once you're done with it, even if an error occurs partway through — you never have to remember to call `file.close()` yourself.
- `file.read()` reads the entire contents of the file into the `content` variable.

2.9.2.2 Writing to Files

```
# Writing to a file (overwrites existing content)
with open("example.txt", "w") as file:
    file.write("As-Salaam-Alaikum, World!\n")

# Appending to a file (adds to the end, keeps existing content)
with open("example.txt", "a") as file:
    file.write("Appending new line.\n")
```

Explanation:

- `"w"` (**write mode**) opens the file and **erases** any existing content before writing new data — use it carefully!
- `"a"` (**append mode**) opens the file and adds new data **after** whatever is already there, without deleting anything.

The Practical Walkthrough:

1. Create `file_handling.py` in the same folder where you'll create your data file.

2. Run the "Writing to a file" code first — this creates `example.txt` with one line of text inside it.
3. Run the "Appending to a file" code next — check `example.txt` again; it should now have two lines.
4. Finally, run the "Opening, Reading, and Closing Files" code to read and print both lines back to the console.

What just happened? You completed the full life cycle of file handling: creating a file with fresh content, adding more content without erasing what existed, and reading everything back — proving the data survived even between separate program runs.

Interactive Stop-Point (Grab a Partner):

Partner A writes a program that asks the user for three of their favorite hobbies, one at a time, and appends each one as a new line into a file called `hobbies.txt`. Partner B then writes a separate program that opens `hobbies.txt` in read mode and prints its full contents. Run Partner A's program first, then Partner B's — does the data persist correctly?

Quick Recap (2.9.2):

`open()` with mode `"r"` reads a file, `"w"` writes (overwriting existing content), and `"a"` appends (preserving existing content); the `with` statement ensures files are always closed safely and automatically.

2.10 Testing and Debugging in Python

2.10.1 Testing

The Hook (Story Mode):

Before a bridge opens to the public, engineers stress-test it — running trucks over it, checking it under extreme weather, pushing it to its limits — all *before* real people rely on it every day. Programmers do the exact same thing to their code: this is called **testing**.

The Explanation:

Testing is the process of running your code with a variety of inputs to check whether it behaves as expected, aiming to catch problems *before* your program is used in the real world.

2.10.1.1 Types of Testing

| Type | What it checks |
|---------------------|--|
| Unit Testing | Individual pieces of code (like a single function or class) in isolation. Python's <code>unittest</code> module is commonly used for this. |
| Integration Testing | Whether different parts of the code work correctly <i>together</i> . |
| Functional Testing | Whether the software behaves correctly from the <i>user's</i> point of view. |
| Regression Testing | Whether new changes accidentally break existing, previously-working functionality. |

Interactive Stop-Point (Pause & Think):

Imagine you just added a new "Apply Discount" feature to an online shopping cart program. Which type of testing from the table above would specifically confirm that the *existing* "Add to Cart" and "Checkout" features still work correctly after your change? Why?

Quick Recap (2.10.1):

Testing means deliberately running your code under many different conditions to catch problems early — unit, integration, functional, and regression testing each check a different scope of your program's correctness.

2.10.2 Debugging

The Hook (Story Mode):

In 1947, engineers working on the Harvard Mark II computer found their machine behaving strangely. Investigating, computer scientist **Grace Hopper** and her team discovered the actual cause: a real moth trapped inside one of the machine's relays. They taped the moth into their logbook, writing "First actual case of bug being found." From that day forward, the term "**bug**" stuck in computing — and finding and fixing them became known as **debugging**.

The Explanation:

Debugging is the process of finding and fixing errors ("bugs") in your code — identifying their root cause and making the necessary correction. Think of debugging as playing detective: your program's unexpected behavior is the "crime scene," and your job is to follow the clues back to exactly where things went wrong.

Nobody, not even the most senior professional engineers at the biggest tech companies in the world, writes perfectly bug-free code on the first try. Debugging isn't a sign that you did something wrong as a person — it's simply the normal, everyday rhythm of programming.

2.10.2.1 Common Debugging Techniques

- **Print Statements:** Temporarily add `print()` statements at different points in your code to check the actual values stored in your variables at that moment. Simple, but surprisingly powerful.
- **Debugging Tools:** Use tools like Python's built-in `pdb` (Python Debugger) to pause your program mid-execution, step through it line by line, and inspect variables in real time.
- **Error Messages:** Read your error messages carefully and completely — they almost always tell you the exact line number and the exact type of problem, if you take the time to read them rather than panic and skip past them.

The Practical Walkthrough — A Debugging Detective Story:

1. Create `buggy_code.py` and deliberately type this broken code:

```
```python
def calculate_average(scores):
 total = sum(scores)
 average = total / len(scores)
 return average
```

```
student_scores = [85, 90, 78, 92]
```

```
print("Average score:", calculate_averge(student_scores)) # typo!
```

```
```
```

2. Run it. Python will raise a `NameError` because `calculate_averge` is misspelled.
3. Read the error message carefully — it tells you the exact line number where the problem occurred.
4. Fix the typo (`calculate_average`) and run again.
5. Now, deliberately pass an empty list, `calculate_average([])`, and run once more.
6. You'll now get a `ZeroDivisionError`, because `len([])` is 0. Add a `try-except` block (from Section 2.9.1) around the function call to handle this gracefully.

What just happened? You experienced the real debugging cycle: run the code, read the exact error Python gives you, form a hypothesis about the cause, fix it, and re-test — repeating until the program behaves correctly in every case you can think of.

Interactive Stop-Point (Grab a Partner):

Partner A writes a short program (5–10 lines) with one deliberate, hidden bug — a typo, a wrong operator, a missing colon, anything at all — and gives it to Partner B without saying what's wrong. Partner B must read the error message (or trace the incorrect output) and identify the bug within two minutes. Then switch roles.

Quick Recap (2.10.2):

Debugging is the normal, healthy process of locating and correcting bugs, using techniques like print statements, dedicated debugging tools, and — most importantly — carefully reading error messages, which are Python's honest, helpful clues rather than a judgment on your ability.

Chapter Summary — The Big Picture

Let's zoom back out. In this unit, you built up an entire programming toolkit, piece by piece:

- You learned that **programming** means giving a computer precise instructions, and you set up your own **development environment**.
- You learned to store data in **variables**, following Python's naming rules, across four core **data types**.
- You learned to gather information with `input()` and display it with `print()`, and to convert text into real numbers with `int()` and `float()`.
- You learned to calculate and compare values using **arithmetic**, **comparison**, **assignment**, and **logical operators**, following a strict **operator precedence**.
- You learned to make your programs think using **if / if-else / if-elif-else** decision structures, and to repeat actions using **while** and **for** loops.
- You learned to organize reusable logic into **functions**, and to borrow powerful pre-built tools using **modules and libraries**.
- You learned to store collections of data using **lists** (flexible, mutable) and **tuples** (fixed, immutable), and to navigate them precisely using **indexing and slicing**.
- You learned to build large programs from small, well-organized pieces through **modular programming**.
- You learned to model real-world things in code using **classes and objects** — the foundation of Object-Oriented Programming.
- You learned to keep your programs running smoothly even when things go wrong, using **try-except exception handling**, and to make your data outlive a single program run using **file handling**.
- And finally, you learned that **testing** and **debugging** are not signs of failure — they are the daily, healthy craft of every single working programmer on Earth.

You are no longer someone who has "never written code." You are a programmer who understands variables, logic, data structures, functions, objects, files, and how to hunt down and fix your own mistakes. That is an enormous amount of real, transferable skill — carry it forward with confidence into Unit 3.

End-of-Chapter Exercises

Multiple Choice Questions

1. An action needed during Python installation to run from the command line easily:
- a) Uncheck "Add Python to PATH" b) Choose a different IDE c) Check "Add Python to PATH"
 - d) Install only the IDE

2. A valid variable name in Python is:

a) `variable1` b) `1variable` c) `variable-name` d) `variable name`

3. Output of the following code:

```
python
age = 25
print("Age: ", age)
```

a) `Age: 25` b) `25` c) `Age` d) `age`

4. The operator used for exponentiation in Python is:

a) `*` b) `**` c) `//` d) `/`

5. A loop used to iterate over a collection such as lists is:

a) `while` b) `for` c) `do-while` d) `repeat`

6. A `range()` function used to generate a sequence of numbers:

a) Generates a list of numbers b) Creates a sequence of numbers c) Calculates the sum of numbers d) Prints a range of numbers

7. A keyword used to define a function in Python:

a) `define` b) `function` c) `def` d) `func`

8. The output of the following code:

```
python
temperature, humidity, wind_speed = 25, 60, 15
print("Hot and humid" if temperature > 30 and humidity > 50 else
      "Warm and breezy" if temperature == 25 and wind_speed > 10 else
      "Cool and dry" if temperature < 20 and humidity < 30 else
      "Moderate")
```

a) Hot b) Warm c) Cool d) Nothing

9. The operation used to combine two lists in Python:

a) `combine()` b) `concat()` c) `+` d) `merge()`

Short Questions

1. Explain the purpose of using comments in Python code.
2. Describe the difference between integer and float data types in Python. Provide an example of each.
3. Define operator precedence and give an example of an expression where operator precedence affects the result.
4. How does the short-hand if-else statement differ from the regular if-else statement?
5. Explain the use of the `range()` function in a `for` loop.
6. Explain how default parameters work in Python functions.
7. Explain why modular programming is useful in Python.
8. Explain the difference between a class and an object in Python.

Long Questions

1. Evaluate the following Python expressions by hand, then verify using real Python code:
 - (a) `(18 / 3 + 4 ** 2) - (2 * (7 - 3)) / (9 % 4)`
 - (b) `(25 + 3 * 4 ** 2 - 6) / (2 ** 3 + 1) - 7`
 - (c) `(12 + 6 * (5 - 2)) ** 2 / ((4 ** 2 - 7) + 10)`
 - (d) `45 / (2 ** 2 + 3 * 4) + 8 * (7 - 3)`
2. Translate the following mathematical expressions into correct Python syntax:
 - (a) $5 \times (3 + 2) / (6 - 2 \times 3)$
 - (b) $7 + 2^2$
3. Explain the concept of variables in Python, using an example of your own.
4. Write a Python program that takes a number as input and checks whether it is positive, negative, or zero using an `if-elif-else` statement.
5. Write a Python program using a `while` loop that prints all the odd numbers between 1 and 100. Also count and print the total number of odd numbers found.